Отчёт по лабораторной работе №4

Обеспечение отказоустойчивости средствами Apache ZooKeeper

1. Цель работы

Освоить развёртывание и настройку распределённого сервиса координации Apache ZooKeeper, а также сформировать понимание механизмов обеспечения отказоустойчивости, автоматического выбора лидера и гарантии согласованности данных в распределённых приложениях. Был поднят отказоустойчивый ансамбль из трёх серверов ZooKeeper и интегрирован с координатором расчёта пар навыков из домашнего задания №3: от проверки кворума и эфемерных узлов до моделирования отказа лидера и демонстрации корректного восстановления с чекпоинта с применением защиты от устаревших записей по эпохе.

2. Этап 1. Установка и подготовка

Для выполнения работы была установлена Java 17 (OpenJDK Temurin 17.0.21) и задана переменная окружения `JAVA_HOME`. Архив Apache ZooKeeper 3.9.6 был распакован в три независимые копии (`zk1`, `zk2`, `zk3`), каждая со своей конфигурацией и папкой данных.

```
$ java -version
openjdk version "17.0.21" 2026-10-20
OpenJDK Runtime Environment Temurin-17.0.21+9
```

Листинг 1: Проверка версии Java

3. Этап 2. Ансамбль ZooKeeper

3.1. Конфигурация серверов

Для каждой копии был написан файл `conf/zoo.cfg`. Строки участников ансамбля одинаковые, а порты и `dataDir` у каждой свои, чтобы серверы не перезаписали данные друг друга. В `dataDir` каждой копии был положен файл `myid` с номером сервера.

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=zk1/data
clientPort=2181
admin.serverPort=8081
server.1=localhost:2888:3888
server.2=localhost:2889:3889
server.3=localhost:2890:3890
```

Листинг 2: Конфигурация zk1/conf/zoo.cfg

Для `zk2` и `zk3` изменяются только `dataDir`, `clientPort` и `admin.serverPort` (2182/8082 и 2183/8083 соответственно).

3.2. Проверка ролей и отказоустойчивости

После запуска трёх серверов штатной командой `zkServer` был определён лидер и два последователя. Далее были проведены тесты отказоустойчивости.

Сценарий А: потеря 1 сервера из 3. Кластер продолжает работать, кворум (2 из 3) сохраняется.



Рис. 1: Успешное подключение клиента после остановки одного сервера

Сценарий Б: потеря 2 серверов из 3 (50%). Кворум потерян, кластер недоступен.



Рис. 2: Ошибка `KeeperErrorCode = ConnectionLoss` при недоступности 2 из 3 серверов

Вывод: Экспериментально подтверждено, что ансамбль из 3 серверов ZooKeeper переживает потерю до 1 узла (кворум 2 из 3), но теряет работоспособность при отказе 2 узлов.

3.3. Эфемерный узел

В клиенте `zkCli` был создан эфемерный узел `/test_ephemeral_hw4` с флагом `-e`. После закрытия сессии (`quit`) узел автоматически исчез.

```
WatchedEvent state:SyncConnected type:None path:null zxid: -1
[zk: 127.0.0.1:2181(CONNECTED) 0]    stat /
cZxid = 0x0
ctime = Thu Jan 01 03:00:00 MSK 1970
mZxid = 0x0
mtime = Thu Jan 01 03:00:00 MSK 1970
pZxid = 0x0
cversion = -1
dataVersion = 0
aclVersion = 0
ephemeralOwner = 0x0
dataLength = 0
numChildren = 1
[zk: 127.0.0.1:2181(CONNECTED) 1]    create -e /test_ephemeral_hw4 "hello_zookeeper"
Created /test_ephemeral_hw4
[zk: 127.0.0.1:2181(CONNECTED) 2]    get /test_ephemeral_hw4
hello_zookeeper
[zk: 127.0.0.1:2181(CONNECTED) 3]    ls /
[test_ephemeral_hw4, zookeeper]
[zk: 127.0.0.1:2181(CONNECTED) 4]    quit
```

Рис. 3: Создание эфемерного узла и подтверждение его существования

Вывод: Эфемерные узлы живут только в рамках сессии клиента и автоматически удаляются при её разрыве. Это основа механизма детектора отказа (failure detector) в ZooKeeper.

4. Этап 3. Приостановка лидера без защиты

Были запущены две копии координатора расчёта пар навыков из ДЗ №3 с таймаутом сессии 10 секунд. Координаторы участвуют в выборах лидера через эфемерные последовательные узлы в `/election/leader`. Лидер обрабатывает данные чанками по 5 секунд и после каждого чанка сохраняет чекпоинт в `/calculation/checkpoint`.

4.1. Демонстрация failover с чекпоинтом

Лидер (Coordinator-A) обработал чанки 0-5, после чего был приостановлен. Запасной координатор (Coordinator-B) через ≈10 секунд (таймаут сессии) обнаружил отсутствие лидера, стал новым лидером, прочитал чекпоинт и возобновил расчёт с чанка 5.

Время смены лидера: ≈ 10 секунд (равно `SESSION_TIMEOUT` = 10 с, заданному в условии).

4.2. Двойная запись результата

После завершения расчёта Coordinator-B результат был записан в `/calculation/result`. При последующем перезапуске Coordinator-A (без защиты по эпохе) он также записал результат, увеличив версию узла до 1.

Вывод: Без защиты по эпохе возникает проблема «двойной записи» (split-brain): оба координатора считают себя имеющими право на запись результата, что приводит к неконсистентности данных.

5. Этап 4. Приостановка лидера с защитой

Сценарий был повторён с включённой проверкой эпохи (`FENCING=1`). Каждый координатор при становлении лидером увеличивает счётчик эпохи в `/calculation/epoch` и записывает



Рис. 4: Coordinator-B подхватывает расчёт с чекпоинта (чанк 5) после отказа Coordinator-A

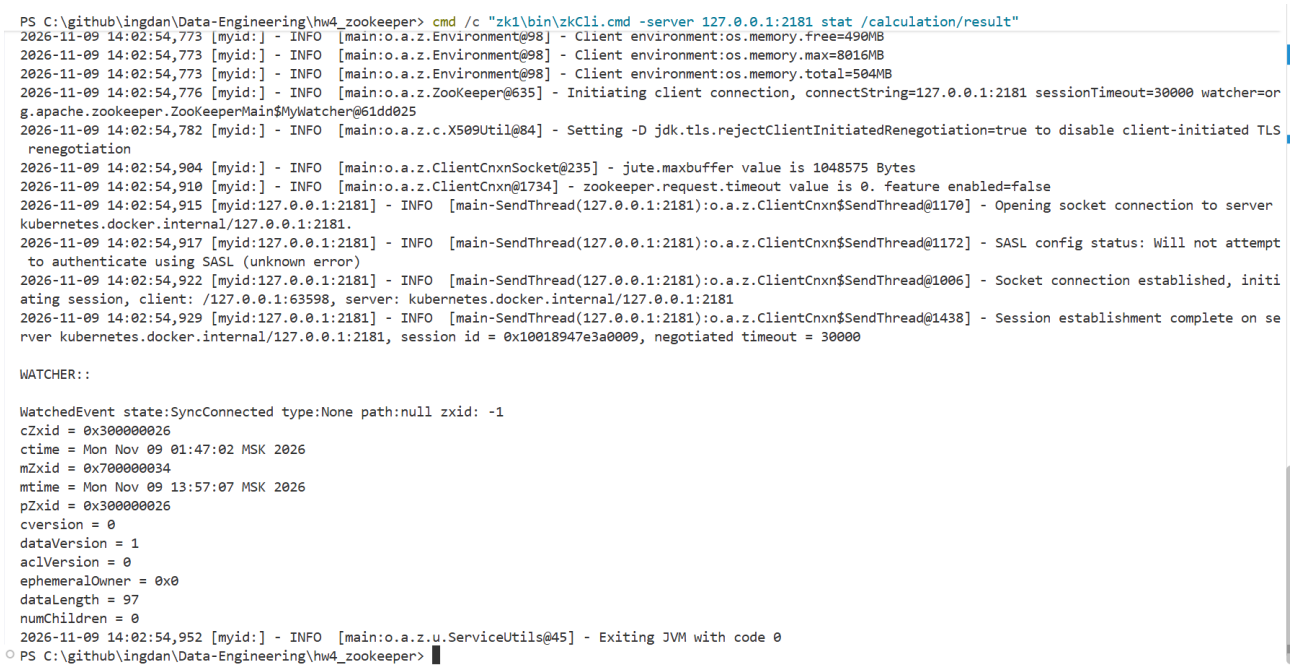


Рис. 5: Обработка чанков и запись результата без защиты (FENCING=0)

свою эпоху в результат.

На скриншоте видно:

- Coordinator-B завершил расчёт и записал результат с эпохой 2;
- Coordinator-A после перезапуска увеличил эпоху до 3, но получил отказ: `Отклонено: результат уже записан координатором Coordinator-B с эпохой 2`.

Вывод: Защита по эпохе (fencing) предотвращает двойную запись: результат зафиксирован единственным координатором, который реально довёл расчёт до конца.

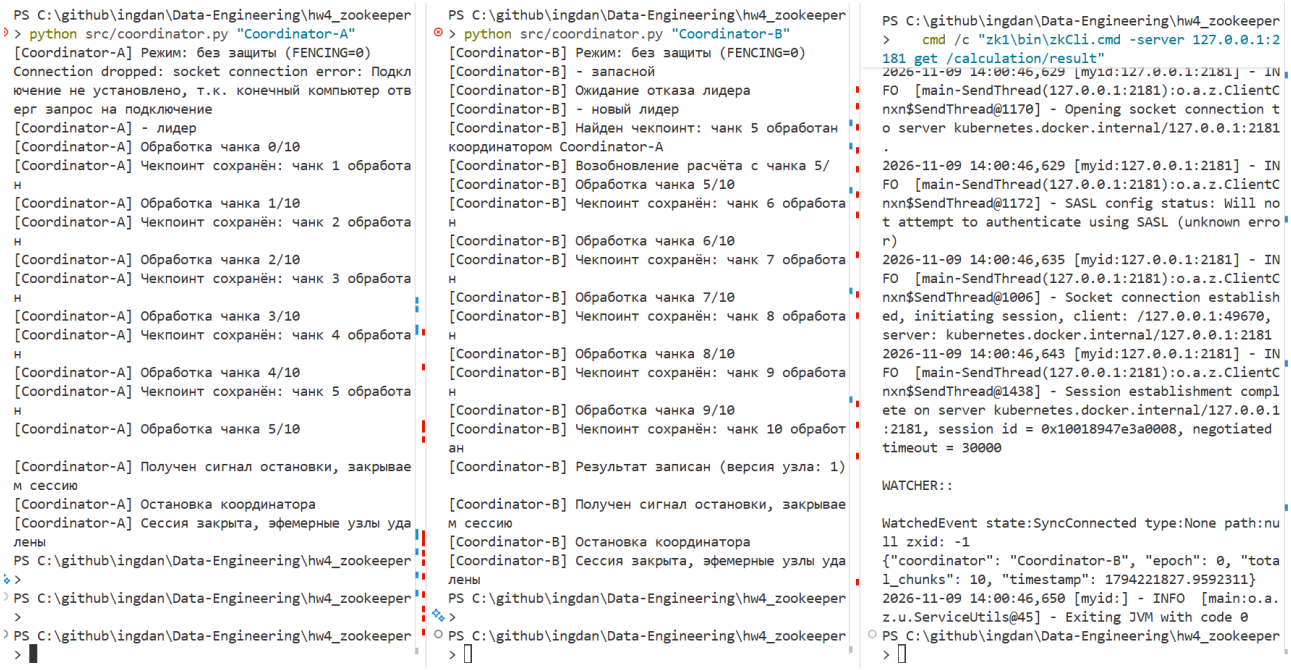


Рис. 6: Статистика узла `/calculation/result`: `dataVersion = 1` (двойная запись)

```
PS C:\github\ingdan\Data-Engineering\hw4_zookeeper> $env:FENCING = "1"
PS C:\github\ingdan\Data-Engineering\hw4_zookeeper> python src/coordinator.py "Coordinator-A"
[Coordinator-A] Режим: с защитой (FENCING=1)
Connection dropped: socket connection error: Подключение не установлено, т.к. конечный компьютер отверг запрос на подключение
[Coordinator-A] - лидер
[Coordinator-A] Найден чекпоинт: чанк 10 обработан координатором Coordinator-B
[Coordinator-A] Возобновление расчёта с чанка 10.
[Coordinator-A] Эпоха увеличена до 3
[Coordinator-A] Отклонено: результат уже записан координатором Coordinator-B с эпохой 2

[Coordinator-A] Получен сигнал остановки, закрываем сессию
[Coordinator-A] Остановка координатора
[Coordinator-A] Сессия закрыта, эфемерные узлы удалены
```

Рис. 7: Coordinator-A получает отказ при попытке перезаписать результат (FENCING=1)

6. Этап 5. Ответы на вопросы

6.1. Почему серверов три, а не два и не четыре?

Три сервера – это минимальное количество для обеспечения **кворума** (строгого большинства) в распределённой системе:

- **2 сервера:** при отказе одного остаётся 1 из 2 – это 50%, что не является большинством. Кворум потерян, кластер не работает. Отказоустойчивость = 0 серверов.
- **3 сервера:** при отказе одного остаётся 2 из 3 – это 67% (большинство), кластер продолжает работать. Отказоустойчивость = 1 сервер.

- **4 сервера:** при отказе одного остаётся 3 из 4 (75%) – работает. При отказе двух – 2 из 4 (50%) – кворум потерян. Отказоустойчивость = 1 сервер (та же, что и у 3).

Вывод: 3 сервера – оптимальный баланс между отказоустойчивостью (переживает отказ 1 узла) и стоимостью. Добавление 4-го сервера не повышает отказоустойчивость, но увеличивает расходы на инфраструктуру.

6.2. Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper?

ZooKeeper – это универсальное хранилище данных, которое предоставляет примитивы (эфемерные узлы, версионирование через `dataVersion`, watchers), но не знает семантику приложения. Он не понимает, что такое «эпоха лидера» или «двойная запись результата».

Получатель записи (координатор) знает бизнес-логику:

- Какую эпоху он представляет;
- Какая эпоха записана в существующем результате;
- Можно ли перезаписывать результат.

Поэтому проверка эпохи реализуется на уровне приложения (в коде координатора), а ZooKeeper только предоставляет механизм атомарной записи с проверкой версии (`set` с параметром `version`), который приложение использует для реализации защиты. ZooKeeper гарантирует атомарность операции, но решение о том, когда применять эту проверку, принимает само приложение.

7. Итоговые выводы

1. Ансамбль из трёх серверов ZooKeeper успешно развёрнут и протестирован: кластер переживает отказ 1 узла, но теряет работоспособность при отказе 2 узлов (потеря кворума);
2. Эфемерные узлы корректно удаляются при разрыве сессии клиента, что является основой механизма failure detection;
3. Координатор расчёта из ДЗ №3 успешно интегрирован с ZooKeeper: реализованы выборы лидера, чекпоинтинг и failover;
4. Без защиты по эпохе (FENCING=0) возникает проблема двойной записи результата (`dataVersion = 1`);
5. С защитой по эпохе (FENCING=1) старый лидер получает отказ при попытке перезаписать результат, зафиксированный новым лидером;
6. Время смены лидера составляет ≈10 секунд и равно таймауту сессии ZooKeeper.